



南開大學
Nankai University

计算机学院
人工智能导论实验报告

实验六：机器人自动走迷宫

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2025 年 5 月 28 日

目录

1 问题重述	2
1.1 实验背景	2
1.2 实验要求	2
1.3 实验环境	2
1.4 注意事项	3
1.5 参考资料	3
2 设计思想	3
2.1 题目一: 实现基础搜索算法	3
2.2 题目二: 实现 Deep QLearning 算法	4
3 代码内容	4
3.1 题目一: DFS	4
3.2 题目二: DQN	6
4 实验结果	9
5 总结	10

1 问题重述

1.1 实验背景

在本实验中，要求分别使用基础搜索算法和 Deep QLearning 算法，完成机器人自动走迷宫。

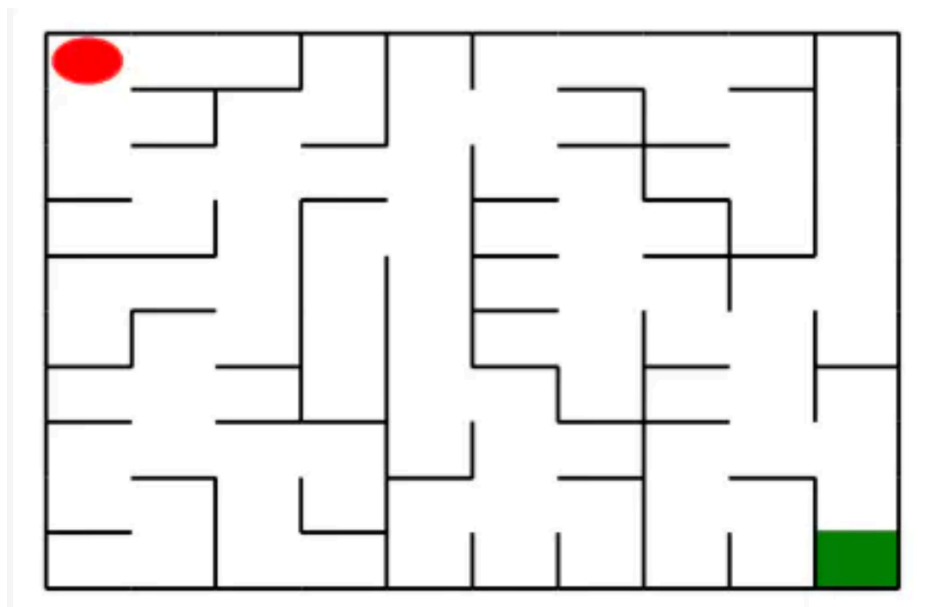


图 1.1: Enter Caption

如上图所示，左上角的红色椭圆既是起点也是机器人的初始位置，右下角的绿色方块是出口。游戏规则为：从起点开始，通过错综复杂的迷宫，到达目标点（出口）。

在任一位置可执行动作包括：向上走'u'、向右走'r'、向下走'd'、向左走'l'。

执行不同的动作后，根据不同的情况会获得不同的奖励，具体而言，有以下几种情况。

- 撞墙
- 走到出口
- 其余情况

你需要实现基于基础搜索算法和 Deep QLearning 算法的机器人，使机器人自动走到迷宫的出口。

1.2 实验要求

- 使用 Python 语言。
- 使用基础搜索算法完成机器人走迷宫。
- 使用 Deep QLearning 算法完成机器人走迷宫。
- 算法部分需要自己实现，不能使用现成的包、工具或者接口。

1.3 实验环境

可以使用 Python 实现基础算法的实现，使用 Keras、PyTorch 等框架实现 Deep QLearning 算法。

1.4 注意事项

- Python 与 Python Package 的使用方式，可在右侧 API 文档中查阅。
- 当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』 - 『Restart Kernel』）。

1.5 参考资料

- 强化学习入门 MDP: <https://zhuanlan.zhihu.com/p/25498081>
- QLearning 简单例子 (英文): <http://mnemstudio.org/path-finding-q-learning-tutorial.htm>
- QLearning 简单解释 (知乎): <https://www.zhihu.com/question/26408259>
- DeepQLearning 论文: <https://files.momodel.cn/Playing%20Atari%20with%20Deep%20Reinforcement%20Learning.pdf>

2 设计思想

2.1 题目一：实现基础搜索算法

我们采用的是深度搜索算法，代码设计如下：

```
1 def my_search(maze):
2     start = maze.sense_robot()
3     root = SearchTree(loc=start)
4     stack = [root] # DFS 使用栈结构
5     h, w, _ = maze.maze_data.shape
6     is_visit_m = np.zeros((h, w), dtype=np.int) # 标记迷宫的各个位置是否被访问过
7     path = [] # 记录路径
8
9     while stack:
10         current_node = stack.pop() # 从栈顶取出当前节点
11         if is_visit_m[current_node.loc]:
12             continue # 如果访问过就跳过
13
14         is_visit_m[current_node.loc] = 1 # 标记为已访问
15
16         if current_node.loc == maze.destination: # 到达终点
17             path = back_propagation(current_node)
18             break
19
20         # 拓展节点
21         if current_node.is_leaf():
22             expand(maze, is_visit_m, current_node)
23
24         for child in current_node.children:
25             stack.append(child)
26
```

27 `return path`

这部分代码设计还是比较简单的，通过参考 BFS 算法，很容易就写出我们的 DFS 算法。函数首先通过 `maze.sense_robot()` 获取起点坐标，并基于该位置构建初始搜索树节点 `SearchTree(loc=start)`，作为根节点加入栈结构 `stack` 中，同时使用 `is_visit_m` 数组标记已访问位置，避免重复搜索与死循环。

在我们的主循环中，每次从栈顶弹出一个节点。若该节点已访问，则跳过；否则标记访问，并判断是否到达终点。若到达终点，通过 `back_propagation` 回溯完整路径并结束搜索。

否则，对于未到终点的叶节点，通过 `expand` 函数生成子节点，并加入当前节点的 `children` 列表里，接着所有子节点随后压入栈中，继续进行深度搜索。

2.2 题目二：实现 Deep QLearning 算法

我们的 Robot 智能体继承自 `MinDQNRobot` 类，实现了训练、动作选择和神经网络搭建等核心功能。训练初期，系统构建了完整的经验池，相当于让智能体“提前看见全图”（金手指），以减少前期的探索难度。这里我参考了 `MinDQNRobot` 类中关于奖励机制的设置，我觉得是很巧妙的一种方法，常规可能是需要找最大 Q 值的行动，但是这里我们设置撞墙 +10 分、每走一步 +1 分，到达终点则给予 -50 分，这样的反逻辑，会促使智能体尽快找到通路，避免频繁的撞墙和多走步子。

模型部分使用 PyTorch 实现，包含两个神经网络：一个用于实时预测（评估网络），另一个用于生成目标值（目标网络）。通过不断从经验池中抽样训练，优化评估网络，并定期同步更新目标网络，从而实现学习过程。

并且我们设计总是根据 Q 值进行选择行动，以加快收敛。

代码部分在下方。

3 代码内容

3.1 题目一：DFS

```
1 import numpy as np
2 # 机器人移动方向
3 move_map = {
4     'u': (-1, 0), # up
5     'r': (0, +1), # right
6     'd': (+1, 0), # down
7     'l': (0, -1), # left
8 }
9
10 # 迷宫路径搜索树
11 class SearchTree(object):
12     def __init__(self, loc=(), action='', parent=None):
13         """
14         初始化搜索树节点对象
15         :param loc: 新节点的机器人所处位置
16         :param action: 新节点的对应的移动方向
17         :param parent: 新节点的父辈节点
18         """
```

```
19
20     self.loc = loc # 当前节点位置
21     self.to_this_action = action # 到达当前节点的动作
22     self.parent = parent # 当前节点的父节点
23     self.children = [] # 当前节点的子节点
24
25     def add_child(self, child):
26         """
27         添加子节点
28         :param child: 待添加的子节点
29         """
30         self.children.append(child)
31
32     def is_leaf(self):
33         """
34         判断当前节点是否是叶子节点
35         """
36         return len(self.children) == 0
37
38
39     def expand(maze, is_visit_m, node):
40         """
41         拓展叶子节点，即为当前的叶子节点添加执行合法动作后到达的子节点
42         :param maze: 迷宫对象
43         :param is_visit_m: 记录迷宫每个位置是否访问的矩阵
44         :param node: 待拓展的叶子节点
45         """
46         can_move = maze.can_move_actions(node.loc)
47         for a in can_move:
48             new_loc = tuple(node.loc[i] + move_map[a][i] for i in range(2))
49             if not is_visit_m[new_loc]:
50                 child = SearchTree(loc=new_loc, action=a, parent=node)
51                 node.add_child(child)
52
53
54     def back_propagation(node):
55         """
56         回溯并记录节点路径
57         :param node: 待回溯节点
58         :return: 回溯路径
59         """
60         path = []
61         while node.parent is not None:
62             path.insert(0, node.to_this_action)
63             node = node.parent
64         return path
65
66     def my_search(maze):
67         """
```

```

68     对迷宫进行深度优先搜索
69     :param maze: 待搜索的maze对象
70     """
71     start = maze.sense_robot()
72     root = SearchTree(loc=start)
73     stack = [root] # DFS 使用栈结构
74     h, w, _ = maze.maze_data.shape
75     is_visit_m = np.zeros((h, w), dtype=np.int) # 标记迷宫的各个位置是否被访问过
76     path = [] # 记录路径
77
78     while stack:
79         current_node = stack.pop() # 从栈顶取出当前节点
80         if is_visit_m[current_node.loc]:
81             continue # 如果访问过就跳过
82
83         is_visit_m[current_node.loc] = 1 # 标记为已访问
84
85         if current_node.loc == maze.destination: # 到达终点
86             path = back_propagation(current_node)
87             break
88
89         # 拓展节点
90         if current_node.is_leaf():
91             expand(maze, is_visit_m, current_node)
92
93         # 注意这里要把子节点逆序压入栈, 保持和广度一样的方向顺序
94         for child in current_node.children:
95             stack.append(child)
96
97     return path

```

3.2 题目二: DQN

```

1  # 导入相关包
2  import torch
3  import torch.nn.functional as F
4  from torch import optim
5  from torch_py.MinDQNRobot import MinDQNRobot as TorchRobot # PyTorch版本
6  from torch_py.QNetwork import QNetwork
7
8  class Robot(TorchRobot):
9      def __init__(self, maze):
10         super(Robot, self).__init__(maze)
11         maze.set_reward(reward={
12             "hit_wall": 10.,
13             "destination": -50.,
14             "default": 1.,
15         })

```

```

16         self.maze = maze
17         self.epsilon = 0
18         self.memory.build_full_view(maze=maze)
19
20         self.losses = self.train()
21
22     # 从TorchRobot继承的方法
23     def _build_network(self):
24         seed = 0
25         random.seed(seed)
26
27         """build target model"""
28         self.target_model = QNetwork(state_size=2, action_size=4,
29                                     seed=seed).to(self.device)
30
31         """build eval model"""
32         self.eval_model = QNetwork(state_size=2, action_size=4,
33                                   seed=seed).to(self.device)
34
35         """build the optimizer"""
36         self.optimizer = optim.Adam(self.eval_model.parameters(),
37                                    lr=self.learning_rate)
38
39     def target_replace_op(self):
40         """
41         Soft update the target model parameters.
42         _target = * _local + (1 - )*_target
43         """
44         # 完全替换参数
45         self.target_model.load_state_dict(self.eval_model.state_dict())
46
47     def _choose_action(self, state):
48         state = np.array(state)
49         state = torch.from_numpy(state).float().to(self.device)
50         if random.random() < self.epsilon:
51             action = random.choice(self.valid_action)
52         else:
53             self.eval_model.eval()
54             with torch.no_grad():
55                 q_next = self.eval_model(state).cpu().data.numpy() # use target
56                 model choose action
57             self.eval_model.train()
58
59             action = self.valid_action[np.argmin(q_next).item()]
60         return action
61
62     def _learn(self, batch: int = 16):
63         if len(self.memory) < batch:
64             print("the memory data is not enough")

```



```

61         return
62     state, action_index, reward, next_state, is_terminal =
        self.memory.random_sample(batch)
63
64     """ convert the data to tensor type"""
65     state = torch.from_numpy(state).float().to(self.device)
66     action_index = torch.from_numpy(action_index).long().to(self.device)
67     reward = torch.from_numpy(reward).float().to(self.device)
68     next_state = torch.from_numpy(next_state).float().to(self.device)
69     is_terminal = torch.from_numpy(is_terminal).int().to(self.device)
70
71     self.eval_model.train()
72     self.target_model.eval()
73
74     """Get max predicted Q values (for next states) from target model"""
75     Q_targets_next = self.target_model(next_state).detach().min(1)[0].unsqueeze(1)
76
77     """Compute Q targets for current states"""
78     Q_targets = reward + self.gamma * Q_targets_next *
        (torch.ones_like(is_terminal) - is_terminal)
79
80     """Get expected Q values from local model"""
81     self.optimizer.zero_grad()
82     Q_expected = self.eval_model(state).gather(dim=1, index=action_index)
83
84     """Compute loss"""
85     loss = F.mse_loss(Q_expected, Q_targets)
86     loss_item = loss.item()
87
88     """ Minimize the loss"""
89     loss.backward()
90     self.optimizer.step()
91
92     """copy the weights of eval_model to the target_model"""
93     self.target_replace_op()
94     return loss_item
95
96 def train(self):#在这一部分进行训练，直到走出迷宫
97     losses = []
98     batch_size = len(self.memory)
99
100    while True:
101        loss = self._learn(batch=batch_size)
102        losses.append(loss)
103        self.reset()
104        for i in range(self.maze.maze_size ** 2 - 1):
105            action, reward = self.test_update()
106            if reward == self.maze.reward["destination"]:
107                return losses

```

```
108
109 def train_update(self):
110     state = self.sense_state()
111     action = self._choose_action(state)
112     reward = self.maze.move_robot(action)
113
114     return action, reward
115
116 def test_update(self):
117     state = np.array(self.sense_state(), dtype=np.int16)
118     state = torch.from_numpy(state).float().to(self.device)
119
120     self.eval_model.eval()
121     with torch.no_grad():
122         q_value = self.eval_model(state).cpu().data.numpy()
123
124     action = self.valid_action[np.argmin(q_value).item()]
125     reward = self.maze.move_robot(action)
126     return action, reward
```

4 实验结果

测试详情 展示迷宫 ×

测试点	状态	时长	结果
测试强化学习算法(中级)	✓	0s	恭喜, 完成了迷宫
测试基础搜索算法	✓	4s	恭喜, 完成了迷宫
测试强化学习算法(初级)	✓	3s	恭喜, 完成了迷宫
测试强化学习算法(高级)	✓	429s	恭喜, 完成了迷宫

确定

图 4.2: 实验结果

可以看到四个测试都实现了走出迷宫，证明我们的算法是合理且成功的。

5 总结

通过本次实验，我们完成了 DFS 算法和 DQN 算法实现机器人走迷宫，其中 DFS 算法比较简单。DQN 算法明显难度要大得多，一个是对于奖励机制的设计，另一个是关于神经网络的构建，无论如何，我们最终实现了我们的目标。

与 DFS 算法相比，DQN 算法具有更好的泛化能力，是一种不断学习的算法，可以学习以往的经验，当训练完善之后，对于更大更复杂的迷宫，效果明显会比 DFS 好。而在某些简单或小规模问题中，DFS 更快实现、结果更稳定、无需训练。